CLOUD COMPUTING CAPSTONE

Automated CI/CD, Infrastructure as Code & Multi-Service Container Orchestration

ស្ថាបត្យកម្មកម្រិត Production លើ AWS EC2 ដោយប្រើប្រាស់ GitHub, Jenkins, Terraform, Docker Swarm និង Nginx

---

វគ្គសិក្សា: Advanced Cloud Computing

Repository: https://github.com/Chamroeun12/Assignment | AWS Region: Sydney (ap-southeast-2)

Group1:

SARAN CHAMROEUN

SOM LIHENG

NGY LYTENG

CAPSTONE SCOPE & OBJECTIVES (វិសាលភាព និងគោលបំណងនៃគម្រោង)

Engineering Goals & Rubric Fulfillment (គោលដៅវិស្វកម្ម និងការបំពេញតាមលក្ខខណ្ឌ)

1. GitOps & Version Control

- Feature branching strategy (feature/eks-argocd-cicd)
- Pull Request workflow ជាមួយ Code Reviews និង Automated Hooks
- Commit ញឹកញាប់ ជា Atomic ជាមួយសារច្បាស់លាស់តាម Convention
- គ្មាន Credentials ឬ Secret ណាមួយត្រូវបាន Commit ចូល Repo ឡើយ

2. Infrastructure as Code (IaC)

- បង្កើតហេដ្ឋារចនាសម្ព័ន្ធ AWS EC2 ឡើងវិញបាន ១០០% ដោយ Terraform
- VPC ដាច់ដោយឡែក, Public Subnets, Internet Gateway & Route Tables
- Security Groups សិទ្ធិតិចបំផុត និងផ្តាច់ប្រព័ន្ធខាងក្នុង (self=true)
- ស្វែងរក Canonical Ubuntu 22.04 LTS AMI ដោយ Dynamic Query

3. Automated CI/CD Pipeline

- Declarative Jenkinsfile pipeline ដំណើរការស្វ័យប្រវត្តិនីមួយៗពេល Push
- PEP8 Linting (Flake8 0 errors) និង Pytest (5/5 passed, 89% coverage)
- ស្កេនរកភាពងាយរងគ្រោះ Container Image តាម Trivy CVE Inspector
- Build Image និង Push ទៅកាន់ Amazon ECR ដោយស្វ័យប្រវត្តិ

4. Resilient Container Orchestration

- Docker Swarm multi-node cluster (1 Manager Leader + 2 Workers)
- Nginx Reverse Proxy ជួយ Load-balancing ទៅ Flask Replicas
- Zero-downtime rolling update strategy (order: start-first)
- Extra Credit (+5): ផ្ញើសារដំណឹងស្វ័យប្រវត្តិទៅ Telegram ពីស្ថានភាព Build

SYSTEM ARCHITECTURE (ស្ថាបត្យកម្មប្រព័ន្ធ)

End-to-End Request Path & CI/CD Pipeline Topology (សមាសធាតុប្រព័ន្ធចម្បងៗ)

- **GitHub Repository:** ប្រភពទិន្នន័យចម្បងតែមួយគត់សម្រាប់ Code, Dockerfiles និង Terraform IaC
- **Jenkins CI/CD:** ស្វ័យប្រវត្តិកម្មលើ Testing, CVE Scanning, Push ECR និង Stack Deployment
- **Amazon ECR:** Private Container Registry មាន Encryption AES-256 និងគ្រប់គ្រង Image Version
- **AWS EC2 & VPC:** VPC ដាច់ដោយឡែក 10.0.0.0/16 មាន 1 Manager (t3.small) និង 2 Workers (t3.micro)
- **Nginx Load Balancer:** Reverse Proxy លើ Port 80 សម្រាប់ចែកចាយចរាចរណ៍ទៅកាន់ Flask Replicas
- **Docker Swarm Mesh:** បណ្តាញ Overlay Network (Port 4789) បញ្ជូន Request តាម Internal VIP (backend_app)
- **Telegram Bot Alerts:** ប្រព័ន្ធ Push Notification ជូនដំណឹងភ្លាមៗពី Build Status, Git Tag & សុខភាពប្រព័ន្ធ

INFRASTRUCTURE AS CODE (IaC)

Declarative AWS Provisioning via Terraform (ចំណុចលេចធ្លោនៃការបង្កើតហេដ្ឋារចនាសម្ព័ន្ធ)

⚙ Modular Architecture

រៀបចំកូដជាម៉ូឌុលច្បាស់លាស់តាម main.tf, variables.tf, outputs.tf និង backend.tf ងាយស្រួលថែទាំ

⚙ Dynamic AMI Resolution

ទាញយក AMI តាម data 'aws_ami' 'ubuntu' filter ពី Canonical publisher (099720109477) មិនបាច់ hardcode

⚙ Least-Privilege Security Group

បើក SSH (22) សម្រាប់តែ Admin CIDR, Web (80/443) ជា Public, និង Swarm (2377, 7946, 4789) សម្រាប់ self=true តែប៉ុណ្ណោះ

⚙ Multi-Node Compute Topology

រៀបចំ 1 Swarm Manager (t3.small) និង 2 Workers (t3.micro) ភ្ជាប់ជាមួយ metadata tags បញ្ជាក់តួនាទី

⚙ Remote State & State Locking

រក្សាទុក Terraform State លើ AWS S3 រួមជាមួយ DynamoDB State Locking ដើម្បីការពារ concurrent mutation

CONTAINERIZATION STANDARDS (ស្តង់ដារៀបចំ CONTAINER)

Production Dockerfile & Security Hardening (ការពង្រឹងសុវត្ថិភាព Dockerfile)

📦 Multi-Stage Build Pattern

ញែកដាច់រវាង Dependencies សម្រាប់ Build Python និង Runtime ស្រាលបំផុត python:3.10-slim

📦 Non-Root Security Context

ដំណើរការដោយ appuser (UID 1000) ជាមួយ chown -R appuser:appuser /app ទប់ស្កាត់ការទម្លាយ container

📦 Container Healthcheck

ប្រើ Native Docker HEALTHCHECK ផ្ទៀងផ្ទាត់ endpoint http://127.0.0.1:5000/health រៀងរាល់ 30s (retry 3 ដង)

📦 Local Multi-Service Compose

ឯកសារ docker-compose.yml សម្រាប់ developer test លើម៉ាស៊ីនផ្ទាល់ មាន web និង nginx លើ bridge network

📦 Minimal Image Footprint

ប្រើ .dockerignore ដកចេញនូវ caches, git history និង secrets ដើម្បីកាត់បន្ថយទំហំ និង attack surface

CI/CD PIPELINE AS CODE (JENKINS DECLARATIVE)

Jenkins Declarative Pipeline — 100% Green Build #4 (ដំណាក់កាលនៃ CI/CD)

✅ **Stage 1: Checkout SCM**

ទាញយកកូដ commit ចុងក្រោយពី GitHub (Chamroeun12/Assignment.git) និងបង្កើត tag កំណែដោយស្វ័យប្រវត្តិ

✅ **Stage 2: Build & Test**

បង្កើត Python venv, ដំណើរការ Flake8 PEP 8 (0 errors) និង Pytest (ជាប់ 5/5 passed, code coverage 89%)

✅ **Stage 3: Security & Quality Scan**

ស្កេនស្រទាប់ Container image រកភាពងាយរងគ្រោះ CVE ដោយប្រើ Trivy vulnerability inspector

✅ **Stage 4: Authenticate & Push ECR**

ផ្ទៀងផ្ទាត់ Docker client តាម aws ecr get-login-password និង push image ជាមួយ tag កំណែ & latest

✅ **Stage 5: Deploy Stack to Swarm**

បញ្ជាពង្រាយ docker stack deploy --with-registry-auth ទៅកាន់ EC2 Swarm cluster ដោយគ្មាន downtime

CONTAINER ORCHESTRATION (DOCKER SWARM)

Docker Swarm Multi-Node Topology & Rolling Updates (ស្ថាបត្យកម្ម Swarm Cluster)

Cluster Formation

1 Manager (Leader) គ្រប់គ្រង Raft consensus (2377/tcp) និង 2 Worker nodes ក្នុងស្ថានភាព Ready & Active

Routing Mesh & Overlay Network

VXLAN overlay (4789/udp) សម្រាប់ភ្ជាប់ multi-host container និងមាន VIP load balancing ខាងក្នុងស្រាប់

Zero-Downtime Rolling Update

កំណត់ order: start-first, parallelism: 1, delay: 10s ដោយ container ថ្មីត្រូវ healthcheck ជាប់ទើបបិទ task ចាស់

Credential Forwarding

ប្រើ docker stack deploy --with-registry-auth ដើម្បីឱ្យ worker nodes authenticate ជាមួយ ECR តាមរយៈ manager

Fault Tolerance

Task នឹងត្រូវ reschedule ស្វ័យប្រវត្តិនីមួយៗពេល node ឬ container ជួបបញ្ហា ដើម្បីធានា replica 2/2 រហូត

NETWORK INGRESS & ROUTING (NGINX)

Nginx Reverse Proxy & Round-Robin Load Balancing (ច្រកចូល និងការបញ្ជូនផ្លូវបណ្តាញ)

- 🌐 **Port 80 Ingress Termination:** Nginx ចងភ្ជាប់ជាមួយ Port 80 ការពារ Backend Flask App មិនឱ្យប៉ះពាល់សាធារណៈផ្ទាល់
- 🌐 **Round-Robin Traffic Balancing:** បែងចែកទិន្នន័យទៅកាន់ Internal VIP upstream backend_app (web:5000) ឆ្លងកាត់ Replicas
- 🌐 **GET / (Web UI):** បង្ហាញ Cloud Dashboard ផ្ទាល់ រួមមាន Container Hostname, ស្ថិតិប្រព័ន្ធ និង Runtime
- 🌐 **GET /health (App Health):** ឆ្លើយតប HTTP 200 OK ជាមួយទម្រង់ JSON: {'status': 'healthy', 'version': '1.0.0'}
- 🌐 **GET /api/info (Metadata):** ឆ្លើយតប HTTP 200 OK បង្ហាញ host name, ប្រភេទ cluster (docker-swarm) និង runtime stats
- 🌐 **GET /nginx-health (Proxy Probe):** Endpoint សម្រាប់ពិនិត្យសុខភាព proxy បញ្ជាក់ពីដំណើរការ routing របស់ Nginx

OPERATIONAL OBSERVABILITY (+5 EXTRA CREDIT)

Real-Time CI/CD Notifications via Telegram Bot API (ប្រព័ន្ធជូនដំណឹង Telegram ស្វ័យប្រវត្តិ)

🔔 Automated Post-Action Callbacks

ភ្ជាប់ផ្គាល់ក្នុង Jenkinsfile post { success { ... } failure { ... } } blocks

🔔 Distinct Success & Failure Alerts

ផ្ញើសារទម្រង់ Markdown បង្ហាញ Job Name, Build #, Git SHA, និង Target Cluster

🔔 Secure Credential Injection

Bot token និង Chat ID ទាញយកតាម Jenkins withCredentials ការពារការឆ្លាយលេខសម្ងាត់

🔔 Non-Blocking Resilient Design

គ្រប់គ្រងដោយ try/catch ជាមួយ logging fallback ដូច្នេះ build មិនបរាជ័យឡើយទោះ notification មានបញ្ហា

🔔 Mobile & Desktop Instant Delivery

វិស្វករ DevOps ទទួលបាន push notification ភ្លាមៗលើទូរស័ព្ទត្រឹមតែប៉ុន្មានវិនាទីក្រោយ build ចប់

ENGINEERING RIGOR: CHALLENGES & SOLUTIONS

Key Technical Challenges & Production Solutions (បញ្ហាប្រឈមបច្ចេកទេស និងដំណោះស្រាយ)

1. Swarm Worker ECR Pull Authentication

បញ្ហា: Worker ត្រូវ pull images ពី Amazon ECR ផ្ទាល់ ប៉ុន្តែខ្វះ AWS credentials នៅលើ Manager
ដំណោះស្រាយ: ប្រើ flag --with-registry-auth លើ docker stack deploy ដើម្បី forward token តាម Swarm control plane

2. Zero Downtime During Redeployments

បញ្ហា: លំនាំដើម Docker តែងតែបិទ task ចាស់ចោលមុននឹងបង្កើត task ថ្មី (order: stop-first)
ដំណោះស្រាយ: កំណត់ order: start-first និង parallelism: 1 ជាមួយ container healthcheck ឱ្យ replica ថ្មី ready ទើបបិទ task ចាស់

3. Dynamic AMI Portability Across AWS Regions

បញ្ហា: Hardcoded AMI IDs បណ្តាលឱ្យ error ពេលដំណើរការ Terraform នៅ region ផ្សេង ឬពេល Ubuntu update
ដំណោះស្រាយ: ប្រើ data 'aws_ami' 'ubuntu' filter រក Canonical ID (099720109477) សម្រាប់ Ubuntu 22.04 LTS ចុងក្រោយ

4. Restricting Docker Swarm Cluster Port Exposure

បញ្ហា: ការបើកច្រក Swarm (2377) និង VXLAN (4789) នៅ 0.0.0.0/0 បង្កើតឱ្យមានហានិភ័យសុវត្ថិភាពខ្ពស់
ដំណោះស្រាយ: កំណត់ Terraform Security Group ជាមួយ self=true ផ្តាច់ network អនុញ្ញាតតែ node ក្នុង cluster ដូចគ្នា

DEMONSTRATION & VERIFICATION (ការបង្ហាញផ្ទាល់)

Step-by-Step Live Demonstration for Lecturer (ជំហានបង្ហាញផ្ទាល់ជូនសាស្ត្រាចារ្យ)

💻 ជំហានទី ១: Live Web Application

បើក Browser ទៅ http://16.178.178.44/ដើម្បីពិនិត្យ dashboard ផ្ទាល់លើ Port 80 និង hostname របស់ replica

💻 ជំហានទី ២: HTTP Endpoint Verification

តេស្ត curl -s /health, curl -s /api/info (បង្ហាញ round-robin), និង curl -s /nginx-health

💻 ជំហានទី ៣: Jenkins CI/CD Automation

បើក Jenkins console បង្ហាញ Build # Stage View ពណ៌បៃតងទាំង ៤ ព្រមទាំង Pytest (5/5) & Flake8

💻 ជំហានទី ៤: Docker Swarm Cluster via SSH

SSH ទៅកាន់ Manager node ដំណើរការ 'docker service ls' (2/2 replicas) និង 'docker stack ps' បង្ហាញការចែកចាយ nodes

PROJECT DELIVERABLES & CONCLUSION

All Capstone Requirements 100% Verified & Fulfilled (តម្រូវការ Capstone បំពេញ ១០០%)

- ✅ **Git Repository:** https://github.com/Chamroeun12/Assignment (Clean commit history & feature branches)
- ✅ **CI/CD Pipeline:** Jenkins declarative pipeline មាន ៥ ដំណាក់កាលស្វ័យប្រវត្តិ (All Green in Build #4)
- ✅ **Infrastructure as Code:** Terraform modules ពេញលេញ រួមមាន dynamic AMI, VPC, SGs, និង EC2 nodes
- ✅ **Container Orchestration:** Multi-node Docker Swarm cluster មាន zero-downtime rolling updates & Nginx proxy
- ✅ **Official Deliverables:** ឯកសារ REPORT.pdf & REPORT.docx formatted reports
- ✅ **Extra Credit (+5 Points):** Automated Telegram Bot push notification alerts កំណត់ និងផ្ញើងផ្ទាត់ជោគជ័យ

Thank You! — Questions (សូមអរគុណ!)